

**NORMA DE PADRONIZAÇÃO
DA NOMENCLATURA DE OBJETOS
DO BANCO DE DADOS**

1. Introdução.....	3
1.1 Histórico de Revisão.....	4
2. Regras Gerais.....	5
3. Padrão de nomes (manual de bolso).....	6
4. Padronização dos nomes dos objetos.....	7
4.1 Banco de Dados.....	7
4.1.1 Bancos do Servidor de Habilitação.....	7
4.1.2 Bancos do Servidor de Veículo.....	7
4.2 Tabelas.....	7
4.3 Colunas.....	9
4.3.1 Colunas nos bancos do Servidor de Habilitação.....	10
4.3.2 Colunas nos bancos do Servidor de Veículos.....	10
4.3.3 Tabela de Referência para Siglas dos Dados.....	11
4.3.4 Chaves Primárias (pk).....	12
4.3.5 Chaves Estrangeiras (fk).....	12
4.3.6 Exceção na Nomenclatura de Colunas - FK.....	13
4.4 Nomes de Constraints.....	14
4.4.1 Primary key.....	14
4.4.2 Foreign key.....	14
4.4.3 Unique.....	14
4.4.4 Check.....	14
4.5 Nome de View.....	15
4.6 Índice.....	16
4.7 Procedures.....	16
4.7.1 Exceções dos Sistemas.....	17
4.7.2 Prefixos Especiais.....	17
4.7.3 Variáveis em Procedures.....	18
4.7.4 Estrutura Padrão da Procedure.....	18
4.7.5 Boas Práticas na Procedure.....	19
4.8 Triggers.....	19
5. Tipos de Dados.....	20
5.1 Função de Usuário.....	21
6. Recomendações e boas práticas.....	23
7. Dicionário de Termos.....	26

1. Introdução

O presente documento tem por objetivo propor uma forma de padronização para os nomes de objetos gerenciados pelo Sybase, levando em consideração a importância que ela tem para a administração dos dados da empresa no que tange à facilidade de identificação e localização proporcionada.

O documento apresenta os objetos de banco de dados com três itens: regras, sintaxe, e exemplo, isto para facilitar o entendimento dos desenvolvedores, analistas e quem se utiliza deste manual.

Esta proposta está aberta a sugestões que possam melhorá-la tanto no aspecto funcional como no operacional. Funcionalmente, o objetivo é facilitar a recuperação de objetos de um mesmo tipo, com identificação imediata do aplicativo ao qual pertencem. Operacionalmente, devemos nos preocupar com os meios possíveis para controlar a padronização de forma automática.

1.1 Histórico de Revisão

SUMÁRIO DE INFORMAÇÕES DO DOCUMENTO			
Documento: Norma de padronização da nomenclatura de objetos do banco de dados			
Versão	Data	Mudanças	Autor
0.24	10/07/2017	Alteração da versão 0.23	Luciene Santos
0.24	13/07/2017	Inclusão das regras das siglas.	Luciene Santos
0.25	16/08/2018	Ajuste na regra da Trigger	Luciene Santos
0.26	11/11/2019	Colocado o prefixo AutoOperacao_	Luciene Santos
0.27	05/10/2020	Colocação do tipo BigDateTime e BigTime que são DateTime	Luciene Santos
0.28	03/03/2021	Inclusão da regra de índice com mesmo ano de pesquisa	Luciene Santos
0.29	10/11/2025	Atualização nas regras gerais, inclusão do manual de bolso, inclusão da regra de proxy table, inclusão da regra da equipe de infrações, inclusão do tipo Booleano.	Draomiro Aguiar
0.30	DATA	Revisão geral do manual, correção de erros, reformulação de tópicos, inclusão de índice e atualização das regras de padronização.	Geraldo Siqueira

2. Regras Gerais

2.1 Caracteres permitidos

- ✓ Usar apenas letras (A–Z, a–z), números (0–9) e _ (underline).
- ✗ Não usar acentos, cedilha (ç), espaços.
- ✗ Não usar caracteres especiais (#, @, %, \$, !, *, +, -, /, =).

2.2 Forma dos nomes

- ✓ Primeira letra de cada palavra maiúscula (Ex.: ParcelaDebito).
- ✓ Usar termos em português e no singular.
- ✓ Usar nomes curtos, claros e sem ambiguidade.
- ✗ Evitar preposições (Ex.: “de”, “da”, “do”).

2.3 Limite de tamanho

- ✓ Máximo de 30 caracteres (se ultrapassar, usar abreviações coerentes).

2.4 Restrições

- ✗ Não usar palavras reservadas (INSERT, DELETE, SELECT...).
- ✗ Não usar apenas números, verbos ou nomes próprios.

2.5 Siglas

- ✓ Siglas oficiais: primeira letra maiúscula e demais minúsculas (Ex.: Ipva, Cnh).

3. Padrão de nomes (manual de bolso)

Objeto	Padrão	Exemplo
Banco	db + área (até 5 posições) + seq	dbhbio01
Tabela	Singular, claro sem abreviação	Veiculo
Tabela Log	Log + NomeTabela	LogParcelaDebito
Tabela Temp	tmp + NomeTabela	tmpVeiculo
Tabela “z”	z + login + ObjetivoProcedure	zkrmlDuplicidadeDebitoLimpar
Proxy Table	px + Origem + NomeObjeto	pxProtLaudoToxicologico
Coluna	tipo + NomeColuna	nValorPagar, nCpf
PK (Primary Key)	pk + NomeTabela	pkVeiculo
FK (Foreign Key)	fk + TabelaFilha + TabelaPai	fkProcessoUsuario
Unique	u + NomeTabela + Coluna	uUsuarioEmail
Check	chk + NomeTabela + Coluna	chkUsuarioSexo
View comum	vw + NomeTabela	vwUsuarioProcesso
View materializada	vm + Objetivo[Complemento]	vmProcessoUsuario
Índice	tabela + _ + Coluna	Usuario_nCpf
Procedure	Objetivo + [Complemento] + Operação	RegistroCfcCategoriaA, AtendimentoE
Trigger	tg + Tabela + Operação	tgTabelaI

4. Padronização dos nomes dos objetos

Como convenção de nomenclatura, o manual adota a combinação de **Pascal Case** e **Notação Húngara** para garantir padronização, legibilidade e identificação rápida dos elementos. Pascal Case consiste em escrever os nomes com a primeira letra de cada palavra em maiúsculo, sem uso de separadores. A Notação Húngara consiste em adicionar prefixos ao nome para indicar o tipo ou finalidade do elemento. Também pode ser utilizada para identificar o tipo de objeto.

4.1 Banco de Dados

O nome do banco de dados deverá identificar o negócio que está sendo automatizado ou deverá refletir a sigla da aplicação.

Nome do banco: dbaaaaann

Onde: **db** → define que é uma database

aaaaa → define a área, o negócio (até 5 posições)

nn → define um sequencial

Exemplo: dbhbio01, dbhpop01, dbimp01

4.1.1 Bancos do Servidor de Habilitação

dbapoio01, dbdado01, dbept01, dbhbio01, dbhcen, dbhcurso01, dbhedu01, dbhexpg01, dbhfis01, dbhige01, dbhimg01, dbhlog, dbhpop01, dbhseg, dbjur01, dbsrvweb01, dbsup01, dbtom, dbwcept01, Operacao.

4.1.2 Bancos do Servidor de Veículo

dbarrecadacao, dbestoque, dbexpg01, dbfisc01, dbige01, dbimg01, dbimg02, dbimp01, dbinfracao, dblog01, dbrelat, dbrop01, dbtmp01, dbvcen, MPS, Operacao, Prot, RENAVAM.

4.2 Tabelas

Utilização dos nomes completos dos termos que compõem o nome da tabela, excetuando-se quando isto não for possível devido à limitação da quantidade de caracteres que o SGBD impõe (**30 caracteres**). Neste caso, os termos **devem ser abreviados** sem perder a coerência do entendimento. Não abreviar a palavra principal

que compõe o nome de um elemento.

Exemplo: ControleLicencaBiometriaCredenciado

Poderia ficar: ControleLicencaBioCredenciado

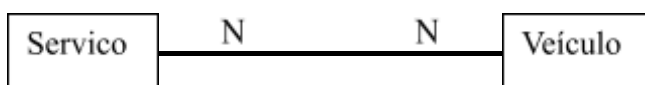
Poderia ficar: ControleLicencaBiometriaCreden

O nome de uma tabela deverá ser sugestivo. Deve-se fazer o uso de nomenclatura orientado a objeto, por exemplo: se no departamento financeiro for necessário manter uma tabela de feriados, esta tabela deve ser nomeada identificando claramente seu conteúdo, isto significa que seu nome então deverá ser Feriado; **O nome da tabela deve estar sempre no singular.**

Exemplo: “Veiculo” no lugar de “Veiculos”

Quando a tabela for derivada de **relacionamento N:N**, o seu nome deverá ser composto pelos nomes das entidades relacionadas;

Exemplo: ServicoVeiculo



O campo identificador / Chave-Primária deve ser referente ao nome da tabela

Exemplo: pkFeriado

Obs.: Não utilizar id, cod ou semelhante na chave identificadora

Utilização do **1º caracter de cada termo em maiúsculo**, ou seja, primeira letra em maiúscula, demais em minúsculas. Para cada palavra interna, primeira letra em maiúscula, Pascal Case;

Exemplo: ParcelaDebito, MarcaVeiculo, Veiculo

Siglas de Estados (unidades federativas) devem ser representadas com todas as letras em maiúsculo, conforme padrão oficial.

Exemplo: PE, PB, SP, RJ e etc.

Siglas no geral como Ipva, Cnpj, Cnh e Febraban obedecerá a mesma regra sendo a

primeira letra maiúscula e as demais minúsculas.

Exemplo: Ipva, Cnpj, Cnh, Febraban e etc.

Quando a tabela for de Log, deve-se usar o nome da tabela antecedida do termo “Log”.

Exemplo: LogParcelaDebito

Obs.: Tabelas de Log **não terão** chave estrangeira (fk) no entanto terão chave primária(pk).

Tabela temporária (tmp): É uma tabela que é usada de forma auxiliar, cujo conteúdo será excluído pela própria aplicação.

Exemplo: tmpVeiculo

Tabela do tipo “z”: São tabelas que serão excluídas do banco de dados. Sua nomenclatura deverá ser composta com a letra “z” no início + login do criador + nome da tabela.

Exemplo: zgcsAlunoTurma

Tabela do tipo Proxy: As Proxy tables são tabelas “espelho” ou de referência que permite acessar dados que estão em outro banco de dados ou servidor, como se estivessem no banco local. Sua nomenclatura deverá ser composta da seguinte forma:

Estrutura: px + Origem + NomeDoObjeto

Exemplo: pxProtLaudoToxicologico

4.3 Colunas

Os nomes das colunas devem ser **claros, descritivos e coerentes com o dado armazenado**, utilizando termos completos sempre que possível. A abreviação de termos deve ser evitada, exceto quando necessária por limitação técnica.

Assim como nas tabelas, o nome da coluna deve representar o mais próximo possível do seu objetivo / significado, facilitando a leitura e entendimento do modelo de dados.

O nome de uma coluna será composto por um prefixo que identifica o tipo do dado,

em **letra minúscula**, seguido pelo nome identificador da coluna, iniciando cada termo com **letra maiúscula**.

Estrutura: tipo do dado + NomeIdentificador

Exemplos: nValorPagar, sNomeCondutor, dDataEmissao.

Siglas como Ipva, Cnpj, Cnh e Febraban obedecerá a mesma regra sendo a primeira letra maiúscula e as demais minúsculas.

ATENÇÃO: A utilização do prefixo de tipo de dado depende da **base de dados à qual a tabela pertence**. A regra segue:

4.3.1 Colunas nos bancos do Servidor de Habilitação

Nas tabelas pertencentes ao banco de Habilitação, **deve-se utilizar o prefixo** que identifica o tipo do dado antes do nome da coluna.

4.3.2 Colunas nos bancos do Servidor de Veículos

Nas tabelas pertencentes ao banco de Veículos, **não deve ser utilizado o prefixo** de tipo de dado antes do nome da coluna.

Como **exceção**, quando a tabela pertencer ao banco **dbfisc**, o prefixo do tipo de dado deverá ser utilizado.

Exemplos:

Significado da Coluna	Banco Habilitação + dbfisc	Banco Veículos
Nome	sNome	Nome
CPF	nCpf	Cpf
Data de nascimento	dDataNascimento	DataNascimento

Se a coluna representar um **campo identificador**, deve-se utilizar a letra “i” em minúsculo na segunda posição do nome, logo após o prefixo do tipo de dado.

Exemplo: niPagamento;

Onde: n → tipo do dado, i → campo identificador,

Pagamento → significado do campo;

Nesse exemplo, indicativo do Pagamento, poderia ser: 1 = On-Line, 2 = Manual.

No caso das tabelas que possuem procedures ligadas a elas (**external procedure**) as colunas que serão usadas como passagem de parâmetros deverão iniciar seu nome com underline.

Exemplo: _sNome (Habilitação)
_Nome (Veículo)

As **tabelas de Log** que são uma réplica de uma tabela do sistema já existente terão a constituição dos nomes das suas colunas idênticas aos da tabela já existente. No caso das colunas que não existam na tabela do sistema a nomenclatura dessas colunas deverá seguir o padrão já existente.

Exemplo: VeiculoChassi ➔ LogVeiculoChassi

4.3.3 Tabela de Referência para Siglas dos Dados

Tipo do Dado	Sigla do Tipo
bit, booleano	b
datetime, smalldatetime, bigdatetime, bigtime	d
text, image, binary, long	I
money, smallmoney	m
numeric, int, smallint, tinyint, float, real, double precision	n
char, varchar	s
Time	T

4.3.4 Chaves Primárias (pk)

Sempre que possível, deve-se utilizar atributos que já **identificam naturalmente** o registro, acrescidos do nome da tabela. Como nos exemplos abaixo:

Exemplos:

Tabela ➡ Profissional

Chave Primária ➡ nCpfProfissional

Tabela ➡ Ofício

Chave Primária composta ➡ nAnoOfício / nNumOfício

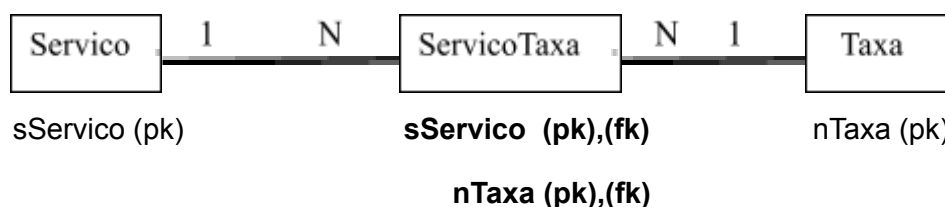
Se a chave primária for um **número sequencial (Cod)**, deve-se utilizar o nome da tabela como nome da coluna que identifica.

Exemplo: Tabela ➡ Processo;

Chave Primária ➡ nProcesso;

Quando a tabela for de relacionamento, os nomes dos campos que compõem a chave primária devem ser mantidos conforme definidos nas tabelas de origem, preservando a nomenclatura original dos campos importados. Esses campos atuarão simultaneamente como chave primária e chave estrangeira;

Exemplo:



4.3.5 Chaves Estrangeiras (fk)

Deve-se utilizar o **mesmo nome** da coluna da tabela de origem (tabela pai);

Exemplo:



Se o campo **não for chave primária** na tabela de origem (tabela pai), deve-se acrescentar o nome da tabela de origem (tabela pai) ao nome do campo.

Exemplo:



4.3.6 Exceção na Nomenclatura de Colunas - FK

Em tabelas que concentram dados de múltiplas origens (ex.: tabelas de integração, log, consumo ou consolidação), pode haver **conflito semântico ou ambiguidade de nomes**.

Quando houver necessidade de uma coluna estrangeira ser criada mais de uma vez em uma tabela deve-se:

- Acrescentar ao nome da coluna o **nome da tabela de origem (tabela pai)**
- Manter o nome original como base
- Garantir clareza e identificação do vínculo

Exemplo:

Considere a seguinte situação:

A tabela **Processo** possui um relacionamento com a tabela **Setor**

A chave é composta por: Setor + Processo

Ao criar uma tabela de destino (ex.: integração ou consumo), pode ocorrer o seguinte cenário:

A tabela pode ter **mais de um campo "Setor"**. Um relacionado ao **Processo** Outro relacionado a **outra entidade (ex.: Atendimento, Unidade, etc.)**

Origem:

Tabela: Processo

Colunas: Setor

Processo

Tabela destino (integração):

Colunas: SetorProcesso

Processo

Essa regra deve ser aplicada quando:

- Existir possibilidade de **colunas com mesmo nome e significados distintos**
- Houver risco de **ambiguidade ou perda de entendimento**

4.4 Nomes de Constraints

4.4.1 Primary key

Usar o prefixo “pk” mais o nome da tabela.

Estrutura: pk + NomeTabela

Exemplo: pkVeiculo

4.4.2 Foreign key

Usar o prefixo “fk” mais os nomes das tabelas filha e pai.

Estrutura: fk + TabelaFilha + TabelaPai

Exemplo: fkProcessoUsuario

Quando existir **mais de uma “fk”** para a mesma tabela pai, deve-se usar um número sequencial no final do nome da “fk”.

Exemplo: fkVeiculoCategoriaVeiculo01

4.4.3 Unique

Usar o prefixo “u” mais o nome da tabela mais o nome da coluna.

Estrutura: u + NomeTabela + Coluna

Exemplo: uUsuarioEmail

4.4.4 Check

Usar o prefixo “chk” mais o nome da tabela mais o nome da coluna.

Estrutura: chk + NomeTabela + Coluna

Exemplo: chkUsuarioSexo

4.5 Nome de View

Views podem ser adotadas para queries complexas, envolvendo mais de uma tabela, cujo formato de saída dos dados selecionados seja considerado padrão para uma ou mais transações do sistema. O uso de view pode melhorar o tempo de resposta da

consulta, pois o comando SELECT é interpretado no momento de criação da view, e não na sua execução.

Views também podem ser adotadas por questões de segurança, como um instrumento para conceder aos usuários permissão de acesso somente a determinados atributos e/ou a informações já consolidadas.

Views devem ser utilizadas somente para consultas. As tabelas envolvidas em uma view não devem ser atualizadas indiretamente, via comandos de INSERT, UPDATE e DELETE efetuados na view.

Deve-se usar a **mesma semântica** utilizada para as tabelas. Para views não materializadas deve ser prefixada a palavra “vw” seguido do nome da tabela.

Exemplo: vwUsuarioProcesso

View Materializada também deve utilizar a semântica das tabelas.

Onde:

vm: prefixo que identifica uma view materializada.

Objetivo: Usar substantivo que responda a pergunta: O que selecionar?
Muitas vezes coincidirá com o nome da tabela.

Complemento:

Opcional. Caso o objetivo não seja suficiente para traduzir a função da view materializada acrescenta-se um complemento / descrição para melhor definição da mesma.

Estrutura: vmObjetivo[Complemento]

Exemplo: vmProcessoUsuario

4.6 Índice

Nome da tabela mais underline seguido do nome da primeira coluna que compõe o índice. Usar abreviações se o nome do campo for muito grande.

Estrutura: Tabela_PrimeiroCampo

Exemplo: Usuario_nCpf

Quando existir mais de um “índice” com o mesmo campo, deve-se usar um número sequencial no final do nome do “índice”.

Exemplo: TipoObjeto_nObjeto01

4.7 Procedures

Estrutura para nome de procedure: Objetivo[Complemento]Operacao.

Onde:

Objetivo: Usar substantivo que responda às perguntas: O que selecionar? O que inserir? Neste caso, muitas vezes coincidirá com o nome da tabela.

Complemento: Opcional. Caso o objetivo não seja suficiente para traduzir a função da procedure, acrescenta-se um complemento / descrição para melhor definição da mesma.

Operação: Usar sigla, em letra maiúscula, das operações básicas:

- Selecionar (S);
- Inserir (I);
- Excluir (E);
- Alterar (A);
- Relatório (R).

Exemplo: RegistroCfcCategoriaA

AtendimentoE

Quando a operação executada **não estiver entre as operações citadas anteriormente**, deve-se utilizar o verbo correspondente por extenso, no infinitivo, ao final da nomenclatura no padrão Objetivo[Complemento]**Operação**.

Exemplo: TurmaCandidato**Vincular**

ApreensaoVeiculo**Gerar**

Siglas como IPVA, CNPJ, CNH, FEBRABAM, AutoOperacao_, obedecerá a mesma regra sendo a primeira letra maiúscula e as demais minúsculas.

Exemplo: Ipva, Cnpj, Cnh, Febrabam, AutoS_Acessologado e etc.

4.7.1 Exceções dos Sistemas

- **Habilitação (RENACH)**

Manter o padrão já existente das procedures do RENACH.

Exemplo: RenachProcessoAtualizar

- **FISEPE (SEFAZ)**

Devem iniciar com as letras "FI" (maiúsculo).

Exemplo: FITipoDebitoS

- **Veículos (RENAVAM/RENAINF)**

As procedures manterão os padrões já existentes internamente no banco. Em outros bancos, só utilizar nomenclatura relacionada se houver vínculo com o projeto.

Exemplo de padrões do RENAVAM:

DetranResp206; PartResp206; **rnv**DetranResp206

Outros Bancos:

dbvcen..**bin**ConsultaVeiculoS_EV02;

dbvcen..Transacao**bin**920;

dbinfracao..**rnf**CodRetornoS.

4.7.2 Prefixos Especiais

Utilizar os seguintes prefixos nas respectivas situações:

Situação	Prefixo	Exemplo
Processamento batch	Batch	BatchProcessoGerar
Integração entre sistemas (RPC)	Rpc	RpcProcessoAtualizar
Acesso via internet	i (minúsculo)	iProcessoAtualizar

Obs.: Não existe prefixo "tmp" para procedures, mesmo que elas alterem tabelas temporárias.

Procedure do tipo "z": O nome da procedure deve ser composto da seguinte forma:
z + login + objetivo da procedure (não podemos colocar o nome da tabela, pois

normalmente esse tipo de procedure não afeta apenas uma tabela)

Estrutura: z + login + objetivo da procedure

Exemplo: zkrmlDuplicidadeDebitoLimpar

(z + krml + DuplicidadeDebitoLimpar)

4.7.3 Variáveis em Procedures

Nas procedures de **acesso à Internet** os nomes das variáveis que são nomes de colunas nas suas respectivas tabelas, deve ser igual ao nome da coluna antecedido do @.

Exemplo: @nTaxa

Para variáveis que **não são nomes de colunas**, deve-se colocar o @ e seguir as mesmas regras utilizadas para nomeação de colunas.

Exemplo: @nValorAux

4.7.4 Estrutura Padrão da Procedure

Na Procedure definir uma área de identificação geral, onde deverão existir informações seguir o padrão abaixo:

Fazer uma descrição clara e objetiva da função da procedure. Colocar a data de criação e o responsável pela criação da procedure (quem alterar a procedure deve acrescentar o nome do responsável pela alteração, a data e o objetivo da alteração);

Exemplo:

```
CREATE PROCEDURE ProcessoS
/*****
** CRIADA POR: Nome do desenvolvedor
** DATA: 01/08/03
** OBJETIVO: Seleciona processos cadastrados a partir de uma determinada data.
**
*****/
@dAbertura smalldatetime, // Cadastramento do Processo
```

@nProcesso numeric(13,0) = 0 // Número do Processo

...

4.7.5 Boas Práticas na Procedure

Identificar as alterações efetuadas na área apropriada, para permitir fácil localização de problemas no código.

Indentar o código de modo a tornar a codificação clara e facilitar o trabalho de manutenção, pode-se usar para indentar a ferramenta SQL Expert.

Ao longo da procedure, **inserir comentários** sempre que necessário. Comentários adicionais que auxiliem a compreensão de problemas complexos. Não poluir o código com comentários desnecessários, que descrevem procedimentos óbvios.

Utilização das palavras reservadas SQL em letras maiúsculas, inclusive os tipos de dados (WHERE, FROM, NUMERIC, etc...).

Quando houver join, as colunas devem ser prefixadas de preferência com a primeira letra do nome da tabela.

Tratando-se de um nome de tabela composto, usar as primeiras letras das palavras.

Caso seja necessário, use duas, três ou mais letras para diferenciar os alias.

Evitar o aninhamento excessivo de comandos, o que costuma dificultar a manutenção do código. Dar preferência à codificação mais longa, porém mais clara, desde que não prejudique a performance.

4.8 Triggers

Usar o prefixo “tg” seguido do nome da tabela onde está sendo disparada ação e da sigla do evento (I = Inserir, A = Alterar, E = Excluir).

Usar uma trigger para cada evento.

Exemplo: tgTabelaI

tgTabelaA

5. Tipos de Dados

Tipos de dados	Faixa de valores	Espaço ocupado
Bit	0 ou 1	1 byte
Int	- 2.147.483.648 a 2.147.483.647	4 bytes
Smallint	- 32.768 a 32.767	2 bytes
Tinyint	0 a 255	1 byte
Text		Até 2.147.483.647 bytes
Image		Até 2.147.483.647 bytes
Char(n)	Range igual à página do servidor 2K = 1948 caracteres (lock allpage) e 1954 (lock datarow)	N bytes
Varchar(n)	Range igual à página do servidor 2K = 1948 caracteres (lock allpage) e 1954 (lock datarow)	N bytes
Numeric(p,s)	- 10E38 a 10E38 - 1	Depende da precisão (P)
Datetime	1/1/1753 a 31/12/9999 precisão: milissegundos	8 bytes
Smalldatetime	1/1/1900 a 6/6/2079 precisão: minutos	4 bytes
Float	Depende da máquina	4 ou 8 bytes
Double precision	Depende da máquina	8 bytes
Real	Depende da máquina	4 bytes
Binary(n)	Range igual à página do servidor 2K = 1948 caracteres (lock allpage) e 1954 (lock datarow)	N bytes
Money	922.337.203.685.477,5807 a 922.337.203.685.477,5807	8 bytes
Smallmoney	214.748.3648 a 214.748.3648	4 bytes

Atributos de Uso Comum:

Atributos	Tipos de Dados
Pessoas	Varchar(50)
E-mail	Varchar(60)
Telefone	Varchar(10)
Fax	Varchar(10)
Logradouro	Varchar(60)
Complemento	Varchar(65)
CEP	Numeric(8)
Bairro	Varchar(60)

Município	Varchar(60)
País	Varchar(60)
CGC	Char(14)
CPF	Char(11)
Login	Varchar(30)

5.1 Função de Usuário

As funções de usuário devem seguir um padrão de nomenclatura que identifique claramente seu escopo de utilização e finalidade.

Estrutura do Nome: sp_f_[Escopo]_Objetivo[Complemento]

Onde:

sp_f → identifica função de sistema

Escopo → define se a função é global ou específica de banco

Objetivo → substantivo que indica a finalidade da função

Complemento → opcional, caso o objetivo não seja suficiente para traduzir a finalidade da função, acrescenta-se um complemento

Prefixo sp_f identifica uma função para manipular strings e/ou números ou objetos de sistema, que são comuns a todos os bancos.

Tipos de Funções

Função Tipo 1 – Global

sp_f_Objetivo[Complemento] ➡ Função Global, ou seja roda de qualquer banco.

Função Tipo 2 – Específica por Banco

sp_f_Nomedobanco_Objetivo[Complemento] ➡ Roda apenas de um banco de dados específico.

Sugestão: retirar o prefixo db do nome do banco, ex: sp_f_arrec

Obs.:

- **Todas as funções ficarão armazenadas no banco sybssystemprocs**, que é o banco padrão de armazenamento de procedure de sistema, como por exemplo: sp_help, sp_who, etc;
- **O prefixo sp é necessário** para que seja possível a pesquisa da mesma, sem a indicação do banco.

6. Recomendações e boas práticas

Estas diretrizes devem ser seguidas no desenvolvimento de objetos de banco de dados. **Qualquer dúvida** em relação a estas dicas de programação no banco de dados conversar com o **DBA**.

Todo acesso a dados deve ser realizado através de Stored Procedures

Exceção:

INSERT e UPDATE em colunas dos tipos TEXT e IMAGE

Integridade referencial deve ser implementada através de constraints (Primary Key, Unique e Foreign);

Preferencialmente não utilizar a estrutura de programação cursor;

Preservar a cláusula abaixo no script, antes do cabeçalho, a fim de agilizar a criação de procedures e triggers armazenados no repositório.

```
IF EXISTS (SELECT 1 FROM sysobjects WHERE name =
'NomeProcedure')
BEGIN
    DROP PROCEDURE NomeProcedure
END
GO

CREATE PROCEDURE NomeProcedure

IF EXISTS (SELECT 1 FROM sysobjects WHERE name =
'NomeTrigger')
BEGIN
    DROP TRIGGER NomeTrigger
END
```

GO

CREATE TRIGGER NomeTrigger

- Devem ser evitados tipos de dados definidos pelos usuários;

Atributos de quantidade escolher entre os tipos a seguir, dependendo da faixa de valor: TINYINT, SMALLINT, INT;

Atributos de data / hora usar SMALLDATETIME ou DATETIME dependendo da precisão necessária para o cálculo de horas, bem como da faixa de valores de cada;

Atributos com conteúdo alfanumérico com mais de 255 caracteres, usar tipo Varchar, preferencialmente;

Atributos de imagem usar tipo IMAGE;

Evitar join com mais de 4 tabelas, quando for necessário, utilizar tabelas temporárias;

Evitar a utilização de **convert** na cláusula where;

Dar preferência a construções do tipo: SELECT iCol, iCol2 from Table WHERE iCol >= 10 a construções do tipo: SELECT iCol, iCol2 from Table WHERE iCol > 9;

Evitar cláusula NOT EXISTS, NOT IN e NOT LIKE. Usar se possível EXISTS, IN e LIKE;

Varchar x Char: Deve ser aplicado caso a caso. Sendo o dado de tamanho fixo e not null, deve-se usar o tipo char. Outra característica que devemos analisar é se o dado sofrerá muitas atualizações. Caso seja verdade, ele é um forte candidato a ser do tipo char, para se ter ganho de performance. O tipo varchar deve ser utilizado visando

economizar em área de armazenamento;

Not null x Null: Deve ser aplicado caso a caso também, já que o campo null é tratado como tipo variável. Lembrando que em muitas situações podemos usar um valor default, para aplicação do campo not null;

Algumas vezes o sybase não consegue usar as **estatísticas para uma variável** inicializada em tempo de execução, implicando em perda de performance. Então, a forma de resolver esse problema é fazer um split de procedure, que significa dividir a procedure em duas, como o exemplo que segue:

```
DECLARE @sCity varchar(25)
SELECT @sCity = sCity
FROM Publishers WHERE sName = @sName
SELECT sAuName
FROM Authors WHERE sCity = @sCity
```

Split da procedure:

```
CREATE PROCEDURE AuNamesS
    @sName varchar(30)
AS
    DECLARE @sCity VARCHAR(25)
    SELECT @sCity = sCity
    FROM Publishers
    WHERE sName = @sName
    EXEC ProcS @sCity
```

```
CREATE PROCEDURE ProcS
    @sCity VARCHAR(25)
AS
    SELECT sAuName
    FROM Authors
    WHERE sCity = @sCity
```

7. Dicionário de Termos

O dicionário de termos tem por objetivo oferecer um catálogo contendo os termos a serem utilizados para se dar nomes aos objetos de dados do ambiente do Detran-PE contêm palavras, suas abreviaturas e remissões para o uso de sinônimos.

A inclusão dos termos deve ser feita pelos coordenadores das equipes de desenvolvimento de sistemas, apenas eles e a equipe de suporte ao desenvolvimento terão acesso à manutenção deste catálogo.

Antes de incluir um termo, deve ser efetuada uma pesquisa, de forma a certificar-se de que dentre os termos já cadastrados não haja outro com o mesmo sentido que possa ser utilizado.

Os termos homógrafos (que tem a mesma grafia) não deverão ser utilizados, portanto cada termo será sempre utilizado com um, e somente um, sentido.

A seção de suporte ao desenvolvimento é responsável pela validação dos termos incluídos, cabendo a ela a última palavra sobre a utilização ou não de determinado termo ou abreviatura.

Sugestão para criar novas **abreviaturas** para palavra:

Escrever a primeira sílaba e a primeira letra da segunda sílaba.

Exemplo: gramática = gram;
 portugues = port;
 numeral = num.

Se a segunda sílaba iniciar por duas consoantes, escrever as duas.

Exemplo: construção = constr;
 secretário = secr.

Termo	Abreviação	Sinônimo
Data	Data	
Descricao	Desc	Descritivo
		Descritiva
		Descritivo
Dia	Dia	
Quantidade	Quant	
	Quantid	
	Qtd	
	Qtde	
Selecionar	S	Buscar
		Ler
		Pesquisar
		Consultar
		Mostrar

Obs.: Por enquanto não está se utilizando o dicionário de termos para os objetos de dados.